

До сих пор речь шла о работе с уже готовыми таблицами. О создании таблиц упоминалось вскользь и на самом элементарном уровне. Рассмотрим вопросы управления структурами данных подробнее.

За управление структурами данных отвечает раздел SQL, называемый **Data Definition Language - DDL**.

1. Создание таблиц.

Как уже говорилось ранее, для создания таблиц служит инструкция CREATE TABLE:

```
CREATE TABLE [{database.[owner] | owner}.]tableName (  
    columnName dataType [DEFAULT { expression | USER | NULL }]  
        [{ IDENTITY | [NOT] NULL }]  
        [columnConstraint [...]]  
    [, nextColumn [, ...]]  
    [, tableConstraint [, ...]]  
)
```

Для того, чтобы создать таблицу, пользователь должен иметь соответствующие права в той базе, в которой создается таблица. В приведенном синтаксисе команды CREATE TABLE

- **database** - имя базы данных, в которой создается таблица (по умолчанию - текущая база)
- **owner** - имя владельца таблицы (по умолчанию - текущий пользователь)
- **tableName** - имя создаваемой таблицы
- **columnName** - имя поля
- **dataType** - тип данных поля и его размерность
- **columnConstraint** - ограничение уровня столбца
- **tableConstraint** - ограничение уровня таблицы

После указания типа данных и размерности поля могут следовать необязательные ключевые слова, позволяющие задать дополнительные атрибуты поля.

Значение поля по умолчанию задается при помощи ключевого слова **DEFAULT**. После него может быть указано

- Выражение, содержащее константы или вызовы встроенных функций.
- Ключевое слово **USER**, предписывающее использовать в качестве значения по умолчанию имя пользователя, добавляющего запись в таблицу.
- Ключевое слово **NULL**, говорящее о том, что значение по умолчанию не задано. Этот вариант нельзя использовать для столбцов, не допускающих значения **NULL**.

После опции DEFAULT (или сразу после указания типа данных, если значение по умолчанию не указано) может быть указано одно из ключевых слов:

- **IDENTITY** - признак того, что значение поля должно генерироваться автоматически (в таблице может быть только один столбец IDENTITY)
- **[NOT] NULL** - определяет допустимость для поля значений NULL

Пример создания таблицы.

```
CREATE TABLE oper.dbo.Clients (  
    ClientID      numeric(12) IDENTITY, -- ID клиента  
    FirstName     varchar(30) NOT NULL, -- Имя  
    LastName      varchar(70) NOT NULL, -- Фамилия  
    DateBorn      date          NULL,    -- Дата рождения  
    UserCreator   varchar(15) DEFAULT USER NOT NULL /* Имя пользователя,  
                                                    зарегистрировавшего  
                                                    данного клиента */  
)
```

В данном примере в базе **oper** создается таблица **Clients**, принадлежащая пользователю **dbo**. Назначение полей таблицы указано в комментариях. Значение поля **ClientID** будет генерироваться автоматически, поле **UserCreator** по умолчанию будет заполняться логином пользователя, создавшего запись.

Замечание.

Практически любая современная СУБД имеет средства автоматической генерации идентификаторов. В Sybase ASE эта возможность реализована через столбцы **IDENTITY**. Но на практике этой возможностью следует пользоваться осторожно, особенно если данный столбец отвечает за взаимосвязи с другими таблицами.

Например, при неправильной "переливке" данных таблицы, использующей столбец **IDENTITY**, на другой сервер или в другую базу значения таких столбцов могут регенерироваться. Естественно, все ссылки на старые значения окажутся некорректными.

Поэтому столбцы **IDENTITY** лучше либо не использовать вообще, либо использовать для данных с коротким сроком жизни, потеря которых будет не критична.

Для Sybase ASE максимальное число полей переменной длины в таблице - **254**, фиксированной длины - **1024**.

2. Изменение таблиц.

В уже созданные таблицы можно внести изменения при помощи инструкции **ALTER TABLE**. Команда **ALTER TABLE** позволяет:

- добавлять и удалять столбцы и ограничения
- изменять тип данных и размерность столбцов
- изменять признак допустимости значений NULL и значения полей по умолчанию
- изменять схему блокировки таблицы
- разбивать таблицу на разделы и наоборот

Базовый синтаксис ALTER TABLE:

```
ALTER TABLE tableName {  
  ADD columnDefinition [, columnDefinition ...]  
  | DROP { columnName [, columnName ...]  
        | constraintName [, constraintName ...] }  
  | MODIFY columnName [dataType] [[NOT ]NULL]  
        [, columnName [dataType] [[NOT ]NULL] ...]  
  | REPLACE columnName DEFAULT { expression | USER | NULL }  
}
```

здесь **columnDefinition** - блок описания поля, аналогичный инструкции **CREATE TABLE**.

Замечание.

Если в **сохраненной процедуре** (будет рассмотрено позже) используются запросы **select *** из таблицы, структура которой была изменена, то их структура останется прежней, даже если использовалась опция **with recompile**. Нужно пересоздать процедуру, чтобы она «увидела» внесенные изменения.

ASE блокирует доступ ко всей таблице до конца выполнения команды **ALTER TABLE**.

Пример использования ALTER TABLE:

```
-- Добавление полей  
ALTER TABLE Clients  
  ADD Address varchar(120) DEFAULT '' NOT NULL,  
      Phone carchar(13) NULL  
  
-- Изменение значения по умолчанию  
ALTER TABLE Clients REPLACE Address DEFAULT NULL  
  
-- Изменение флага NULL / NOT NULL и типа данных поля  
ALTER TABLE Clients MODIFY FirstName NULL, Address varchar(200)  
  
-- Удаление полей  
ALTER TABLE Client DROP Phone, Address
```

Ограничения и рекомендации.

- Нельзя добавлять столбец типа bit
- Максимальное число полей переменной длины **254**, фиксированной – **1024**
- После удаления полей в таблице должно оставаться как минимум одно поле
- После изменения структуры удостоверьтесь, что все объекты, ссылающиеся на обновленную таблицу, выполняются и выполняются правильно. Используйте **sp_depends**, чтобы определить зависимости между объектами базы данных.
- Преобразование возможно между полями, типы которых допускают явное или неявное преобразование.
- Проверьте реальную длину данных прежде, чем соберетесь уменьшить длину столбца. Уменьшение длины символьного столбца приводит к усечению данных. ASE выдает сообщения об ошибках при усечении данных только, если включена опция **string_truncation**. Реакция ASE при потере точности зависит от установки опции **arithignore arith_overflow**, на потерю разрядности ASE просто не реагирует.

3. Удаление таблиц.

Для удаления таблиц служит инструкция **DROP TABLE**.

```
DROP TABLE tableName [, tableName ...]
```

При удалении таблицы удаляются все данные, хранящиеся в таблице, а также связанные с данной таблицей объекты: ограничения, индексы, триггеры и т.д.

Чтобы удалить таблицу, пользователь должен быть ее владельцем.

Пример:

```
DROP TABLE oper.dbo.Clients
```

DROP TABLE неприменима, если таблица-операнд имеет ограничения ссылочной целостности. Сначала нужно удалить строки из внешней таблицы или удалить внешние ключи.

4. Ограничения.

При создании таблицы можно описывать ограничения, накладываемые как на значение конкретного поля, так и на таблицу в целом или на взаимосвязи данных в различных таблицах. Рассмотренные ранее ключевые слова **NULL / NOT NULL** и **DEFAULT** - это тоже ограничения, хотя и несколько специфические. Их относят к спецификации столбца.

Полный список видов ограничений приведен ниже.

[NOT] NULL	Определяет допустимость для данного поля значения NULL
DEFAULT	Определяет значение поля по умолчанию
PRIMARY KEY	Указывает столбцы, не допускающие NULL, уникально идентифицирующие каждую строку в таблице. Реализуется посредством механизма уникального индекса. В случае составного первичного ключа данное ограничение определяется на уровне таблицы.
UNIQUE	Гарантирует, что столбец или набор столбцов имеют уникальные значения для всех строк таблицы. Допускает значение NULL (одно на таблицу).
FOREIGN KEY ... REFERENCES	Устанавливает связь между таблицами, определяя зависимость между первичными и внешними ключами.
CHECK	Определяет какие данные могут вводиться в определенный столбец, т.е. область значений столбца.

Ограничения используются для предотвращения появления некорректных данных в таблицах и для обеспечения ссылочной целостности.

Синтаксис команды CREATE TABLE поддерживает два варианта описания ограничений: на уровне столбца и на уровне таблицы.

Ограничения, описываемые **на уровне столбца**, располагаются непосредственно в блоке описания конкретного поля и могут влиять только на это поле.

Ограничения, описываемые **на уровне таблицы**, располагаются отдельно и могут ссылаться на несколько полей таблицы.

Общий синтаксис описания ограничений как на уровне столбца, так и на уровне таблицы:

```
[CONSTRAINT constraintName] constraintType [constraintDefinition]
```

Описание ограничения начинается с ключевого слова **CONSTRAINT**, после которого следует имя ограничения. Далее указывается тип ограничения и, возможно, какие-то дополнительные параметры, специфические для данного конкретного типа ограничений.

Ключевое слово **CONSTRAINT** и имя ограничения можно не указывать, в этом случае имя ограничения будет сгенерировано автоматически. Но рекомендуется давать ограничениям осмысленные имена, это может пригодиться при необходимости удалить ограничение.

Ограничения **NULL / NOT NULL** и **DEFAULT** описываются только на уровне столбца. Остальные

ограничения допускают оба варианта описания.

Рассмотрим каждое ограничение более подробно.

PRIMARY KEY.

Практически любая таблица должна содержать поле или несколько полей, комбинация значений которых позволяет однозначно идентифицировать запись. Такое поле или комбинация полей называется **первичным ключом**.

Уникальность комбинации значений полей, входящих в первичный ключ, контролируется на уровне СУБД. Добавить в таблицу две записи с одинаковыми значениями первичного ключа невозможно. Поля, входящие в первичный ключ, не могут содержать значения **NULL**.

Если первичный ключ таблицы состоит из одного поля, то такой первичный ключ можно описать в виде ограничения уровня столбца, если же в первичный ключ входят несколько полей, применяется описание уровня таблицы (первичный ключ из одного поля так же можно описать и на уровне таблицы).

Синтаксис описания ограничения **PRIMARY KEY** на уровне столбца:

```
columnName dataType [CONSTRAINT constraintName]
                    PRIMARY KEY [[NON]CLUSTERED]
```

Синтаксис описания ограничения **PRIMARY KEY** на уровне таблицы:

```
[CONSTRAINT constraintName]
PRIMARY KEY [[NON]CLUSTERED] ( columnName [, ...] )
```

Необязательное ключевое слово **CLUSTERED** или **NONCLUSTERED** позволяет определить тип индекса, который будет использоваться для построения данного первичного ключа (индексы будут рассмотрены позднее). По умолчанию для первичных ключей обычно применяется кластерный индекс, но это можно настроить на уровне параметров сервера.

В конце декларации в круглых скобках через запятую указываются поля, входящие в первичный ключ. Порядок указания полей важен, поскольку он влияет на порядок полей в создаваемом индексе.

Примеры описания первичных ключей:

```
CREATE TABLE Clients (
    ClientID    numeric(12) CONSTRAINT ClientPhonesPK PRIMARY KEY,
    FirstName   varchar(30),
    LastName    varchar(70) )
```

```
CREATE TABLE Clients (
    ClientID    numeric(12),
    FirstName   varchar(30),
    LastName    varchar(70),
    PRIMARY KEY ( ClientID ) )
CREATE TABLE ClientPhones (
```

```

ClientID      numeric(12),
PhoneID       int,
PhoneNumber    varchar(13),
CONSTRAINT ClientPhonesPK
    PRIMARY KEY NONCLUSTERED( ClientID, PhoneID )
)

```

UNIQUE.

Данное ограничение во многом схоже с ограничением **PRIMARY KEY**. Его использование также обеспечивает уникальность значения поля или группы полей в рамках таблицы.

Отличия заключаются в том, что первичный ключ у таблицы может быть только один, а уникальных полей (или их комбинаций) - несколько. Кроме того, поля, на которые наложено ограничение **UNIQUE**, в отличие от полей первичного ключа, могут допускать значения **NULL** (если на них не наложено дополнительное ограничение **NOT NULL**). Индекс, создаваемый для ограничения **UNIQUE**, по умолчанию **NONCLUSTERED**. Ограничение **UNIQUE** может быть использовано, например, для описания альтернативных ключей таблицы.

Синтаксис **UNIQUE** аналогичен синтаксису **PRIMARY KEY**. На уровне столбца:

```
columnName dataType [CONSTRAINT constraintName] UNIQUE [[NON]CLUSTERED]
```

На уровне таблицы:

```
[CONSTRAINT constraintName] UNIQUE [[NON]CLUSTERED] ( columnName [, ...] )
```

Примеры использования UNIQUE:

```

CREATE TABLE Clients (
    ClientID      numeric(12) NOT NULL UNIQUE, -- Описание на уровне столбца
    FirstName     varchar(30),
    LastName      varchar(70)
)

```

```

CREATE TABLE Clients (
    ClientID      numeric(12),
    FirstName     varchar(30),
    LastName      varchar(70),
    UNIQUE ( ClientID ) -- Описание на уровне таблицы
)

```

```

CREATE TABLE ClientPhones (
    ClientID      numeric(12),
    PhoneID       int,
    PhoneNumber    varchar(13),
    CONSTRAINT ClientPhonesUnique          -- Ссылка на несколько полей
        UNIQUE CLUSTERED( ClientID, PhoneID )
)

```

FOREIGN KEY ... REFERENCES.

Данное ограничение позволяет описать взаимосвязи между таблицами с целью поддержания ссылочной целостности. Поле (поля), на которые накладывается данное ограничение, обозначаются как внешний ключ, для которого устанавливается связь с первичным (PRIMARY KEY) или уникальным (UNIQUE) ключом другой таблицы.

Синтаксис данного ограничения таков. На уровне столбца:

```
columnName dataType [CONSTRAINT constraintName]
                        REFERENCES otherTableName( otherColumnName )
```

На уровне строки:

```
[CONSTRAINT constraintName]
  FOREIGN KEY ( columnName [, ...] )
  REFERENCES refTableName [( refColumnName [, ...] )]
```

Ключевое слово **FOREIGN KEY** определяет поля в текущей таблице, чьи значения должны совпадать со значениями полей в родительской таблице или быть **NULL**. **REFERENCES** определяет родительскую таблицу (так же возможны ссылки "сам на себя") и те столбцы в ней, с которыми устанавливается связь. Если столбцы родительской таблицы явно не указывать, считается, что ссылка идет на первичный ключ.

Максимальное число ссылок для таблицы - **192**.

Если вспомнить пример со счетами и клиентами из раздела "Объединение таблиц", то описание этих таблиц с указанием связи может выглядеть так:

```
CREATE TABLE dbo.Clients (
  ClientID    numeric(12) PRIMARY KEY,
  FirstName   varchar(30) NOT NULL,
  LastName    varchar(70) NOT NULL,
  DateBorn    date        NOT NULL
)
CREATE TABLE dbo.Accounts (
  AccountID   char(14)    PRIMARY KEY,
  ClientID    numeric(12) NOT NULL REFERENCES dbo.Clients( ClientID )
)
```

или же внешний ключ для таблицы **Accounts** можно описать на уровне таблицы:

```
CREATE TABLE dbo.Accounts (
  AccountID   char(14)    PRIMARY KEY,
  ClientID    numeric(12) NOT NULL,
  CONSTRAINT FK_Accounts_Clients
    FOREIGN KEY( ClientID ) REFERENCES dbo.Clients( ClientID )
)
```

Следствием создания внешнего ключа будет то, что (в нашем примере) в таблицу **Accounts** станет

невозможно добавить счет, содержащий ID несуществующего клиента, а из таблицы **Clients** невозможно будет удалить клиента, имеющего хотя бы один счет.

CHECK.

Данное ограничение позволяет описать условия, которым должны удовлетворять данные, заносимые в поле таблицы. Условия могут включать

- список значений, заданный оператором **IN**
- диапазон, заданный оператором **BETWEEN**
- условия, определяемые оператором **LIKE**
- арифметические действия и вызовы встроенных функций Transact SQL

При описании условий нельзя использовать подзапросы, переменные или агрегирующие функции.

Ограничения **CHECK**, заданные на уровне таблицы, могут ссылаться на другие столбцы в этой таблице. Ограничения, заданные на уровне столбца, могут ссылаться только на сам столбец. Для каждого поля можно определить несколько ограничений.

CHECK имеет более низкий приоритет по сравнению с **NULL / NOT NULL**. Т.е. его нельзя использовать для того, чтобы запретить ввод **NULL**, если спецификация столбца это допускает.

Пример:

```
CREATE TABLE dbo.Clients (  
    ClientID    numeric(12) PRIMARY KEY,  
    FirstName   varchar(30) NOT NULL,  
    LastName    varchar(70) NOT NULL,  
    DateBorn    date          NOT NULL CHECK( DateBorn < '1980-01-01' )  
)
```

или

```
CREATE TABLE dbo.Clients (  
    ClientID    numeric(12) PRIMARY KEY,  
    FirstName   varchar(30) NOT NULL,  
    LastName    varchar(70) NOT NULL,  
    DateBorn    date          NOT NULL,  
    CONSTRAINT DateBornCheck CHECK( DateBorn < '1980-01-01' )  
)
```

5. Изменение ограничений.

Для уже существующих таблиц ограничения добавляются и удаляются при помощи команды **ALTER TABLE**:

```
ALTER TABLE tableName
  ADD { [CONSTRAINT constraintName]
        { UNIQUE | PRIMARY KEY } (columnName [, columnName ...])
        | FOREIGN KEY (columnName [, columnName ...])
          REFERENCES refTableName [( refColumnName [, refColumnName ...])]
        | CHECK ( checkCondition ) }
  | DROP { CONSTRAINT constraintName
          | PRIMARY KEY
          | UNIQUE ( columnName [, columnName ...] ) }
```

Ограничения **NULL / NOT NULL** и **DEFAULT** добавляются и удаляются при помощи **ALTER TABLE MODIFY** и **ALTER TABLE REPLACE**.

6. Правила (RULE).

Альтернативой ограничениям **CHECK** могут служить правила (**RULE**). В отличие от **CHECK**, правила не являются специфическими для какого-либо столбца или таблицы и могут быть привязаны к столбцам в нескольких таблицах одновременно.

Правила создаются при помощи инструкции

```
CREATE RULE ruleName AS ruleCondition
```

При описании условий правила ссылка на столбец описывается в виде переменной **@col**.

Для того, чтобы использовать правило нужно:

- Создать правило, используя команду **CREATE RULE**.
- Связать правило со столбцом или определяемым пользователем типом данных, используя системную процедуру **sp_bindrule**.

Действие правила можно отключить, вызвав процедуру **sp_unbindrule** или привязав к столбцу новое правило.

Пример.

```
CREATE RULE dateBornRule AS @col < '1980-01-01'  
go  
sp_bindrule dateBornRule "Clients.DateBorn"  
go  
sp_unbindrule "Clients.DateBorn"  
go
```

7. Рекомендации по использованию правил и ограничений.

При использовании правил и ограничений следует помнить, что любая логика проверок, описанная на уровне СУБД, порождает дополнительную нагрузку на сервер баз данных.

В нормальной ситуации логика проверки информации, записываемой в базу данных, реализуется либо в сохраненных процедурах (при использовании двухзвенной архитектура клиент-сервер), либо на уровне сервера приложений (при использовании трехзвенной архитектуры).

Описание строгих правил и ограничений имеет смысл разве что при непосредственном доступе пользователей к БД, что в реальной жизни случается редко.

Обычно использование ограничений заканчивается описанием первичных и внешних ключей (иногда - альтернативных идентификаторов UNIQUE).

Причем, что касается внешних ключей, их описание более уместно в системах-первоисточниках информации. Если же комплекс использует данные, введенные и проверенные другой системой, описание в его БД внешних ключей таблиц может оказаться не только излишним, но и просто вредным.

8. Создание таблиц запросом.

Альтернативным способом создания таблиц является создание таблиц запросом. Синтаксически данный способ является расширением оператора **SELECT** и позволяет создать таблицу, основанную на полях, определенных в списке выбора оператора **SELECT** и наполнить ее данными, выбранными в предложение **WHERE**:

```
SELECT selectList
INTO newTabelName
[FROM fromClause]
[WHERE whereClause]
[...]
```

Команда **SELECT INTO** полезна для создания тестовых таблиц, копий или подмножества существующих таблиц.

Команду можно использовать для создания постоянных таблиц только тогда, когда для базы данных, в которой создается таблица, включена опция "**into/bulkcopy/pilsort**". Временные таблицы (будут рассмотрены ниже) создавать таким образом можно даже если эта опция отключена.

Команда **SELECT INTO** не логируется.

Принципы использования:

- Структура новой таблицы определяется на основании полей в списке выбора **SELECT**.
- Новая таблица сразу наполняется данными, отображенными в соответствии с условиями в разделах **FROM** и **WHERE**.
- Если поле в списке выбора является выражением, для него обязательно должно быть указано имя (путем указания псевдонима).
- Все столбцы новой таблицы, сформированные в результате выполнения функций, отличных от **convert()** или **isnull()**, допускают значения **NULL**.
- Т.к. **SELECT INTO** - не логируемая операция, она не может быть использована внутри транзакций.
- Команда **SELECT INTO** выполняется в два этапа: сначала создается таблица, потом она заполняется данными. Если во время вставки данных произошла ошибка, все вставленные ранее данные остаются в таблице.
- Новая таблица не наследует никаких ограничений таблицы-прототипа.

Примеры:

```
-- Создание копии таблицы (без учета ограничений)
SELECT * INTO ClientsCopy FROM Client
```

9. Временные таблицы.

Временные таблицы служат для хранения короткоживущих оперативных данных. Временные таблицы располагаются в базе **tempdb** (Sybase ASE позволяет создавать несколько "временных" баз данных, но для простоты будем говорить о базе **tempdb**, имея в виду "базу данных для временных таблиц").

Создать временную таблицу может любой пользователь, имеющий права на создание таблиц в **tempdb**, или **dbo**.

Существуют два вида временных таблиц:

- **глобальные**, доступные многим процессам
- **сессионные**, доступные только текущей пользовательской сессии

Для того, чтобы получить **глобальную** временную таблицу, достаточно создать ее в базе **tempdb**:

```
CREATE TABLE tempdb..tempTableName(  
    columnDefinition [, columnDefinition ...] )
```

Созданные таким способом таблицы доступны всем пользовательским сессиям (имеющим соответствующие права). **Глобальные** временные таблицы и данные в них существуют до момента перезагрузки сервера (или до явного уничтожения таблицы при помощи DROP TABLE).

Сессионные временные таблицы доступны только создавшей их пользовательской сессии и по факту закрытия сессии автоматически уничтожаются. Если сессионная временная таблица была создана внутри сохраненной процедуры, она автоматически уничтожается по окончании работы данной процедуры.

При создании **сессионной** временной таблицы указывается имя, начинающееся с #:

```
CREATE TABLE #tempTableName( columnDefinition [, columnDefinition ...] )
```

Созданная таким способом таблица доступна только той сессии, в рамках которой была создана. При этом разные сессии могут одновременно работать с одноименными сессионными временными таблицами, причем с точки зрения сервера это будут разные объекты.

Достигается это за счет того, что сессионной временной таблице на уровне системного каталога СУБД присваивается автоматически сгенерированное уникальное имя. Указанное пользователем название таблицы дополняется 17-символьным суффиксом, уникальным для данной сессии.

Пример:

```
CREATE TABLE #tempTable( field1 int, field2 char(10) )  
SELECT name FROM tempdb..sysobjects WHERE name like '#tempTable%'
```

В результате получим что-то вроде этого:

name
#tempTable____00000090020751973

Вопреки официальной документации, добавляемый суффикс содержит некоторую полезную информацию:

- символы **1-2** (в примере - **00**) указывают уровень вложенности по стеку вызовов, на котором была создана таблица (00 - с клиента, 01 - из процедуры, вызванной с клиента, 02 - из процедуры, вызванной другой процедурой и т.д.)
- символы **3-7** (в примере - **00009**) - это **spid** процесса-владельца таблицы.

Это особенно полезно, когда огромная временная таблица заполняет tempdb. Процесс-владелец может быть идентифицирован по полному имени таблицы и, возможно, удален.

Ограничения для сессионных временных таблиц.

- Нельзя в пределах процедуры создать, удалить и снова создать сессионную временную таблицу с одним и тем же именем
- На таких таблицах нельзя создать триггеры и представления

10. Получение информации о таблицах.

Для доступа к информации о различных объектах базы данных типа таблиц, столбцов, и ограничений ASE предоставляет набор системных процедур и встроенных функций.

Для получения информации о таблицах и других объектах базы данных используется системная сохраненная процедура **sp_help**:

```
[exec] sp_help [@object_name]
```

Процедура **sp_help** возвращает информацию об указанном объекте базы данных (т.е. о любом объекте, присутствующем в **sysobjects**), указанном типе данных (присутствующем в **systypes**), или обо всех объектах и типах данных в текущей базе данных (при вызове без параметров).

Результаты **sp_help** также включают информацию о табличных индексах, включая те, которые созданы посредством ограничений.

Процедура **sp_help** перечисляет, но не описывает ограничения целостности. Для получения информации об ограничениях, связанных с таблицей, используйте процедуру **sp_helpconstraint**:

```
[exec] sp_helpconstraint [@object_name] [, detail]
```

Результаты работы данной процедуры включают в себя информацию о ссылочных ограничениях целостности (referential integrity constraints), включая имя и определение.

При вызове без параметров **sp_helpconstraint** возвращает количество ограничений для каждой таблицы в текущей базе.

При указании опции **detail** в вывод включается информация об ассоциированном сообщении.

Для получения информации о дисковых ресурсах, используемых таблицей, используйте процедуру **sp_spaceused**:

```
[exec] sp_spaceused [@object_name]
```

Процедура **sp_spaceused** вычисляет количество строк и страниц данных, используемых таблицей или кластерным/некластерным индексом.

При вызове без параметра **sp_spaceused** возвращает суммарный объем дискового пространства, используемого всеми объектами базы данных.

Для получения информации об объектах базы данных в табличном виде используйте процедуру **sp_tables**:

```
[exec] sp_tables [@table_name] [, @table_owner]
          [, @table_qualifier] [, @table_type]
```

Можно сказать, что **sp_tables** возвращает список объектов, которые могут появляться в предложении **FROM**.

Параметры:

- **@table_name** – имя таблицы. Допускает групповые символы (для оператора **LIKE**)
- **@table_owner** – владелец таблицы. Допускает групповые символы
- **@table_qualifier** – имя базы данных. Допустимые значения - имя текущей базы данных или NULL
- **@table_type** – тип отображаемых объектов ('TABLE', 'SYSTEM TABLE', 'VIEW' - именно так, апострофы в данном случае - часть значения, а не ограничители строки)

Также информацию об объектах базы можно получить, сделав выборку из таблицы **sysobjects**. Поле **type** содержит тип объекта:

- U - пользовательская таблица
- S - системная таблица
- P - сохраненная процедура
- T - триггер
- V - представление (VIEW)

Для получения информации о полях таблицы в табличном виде используйте процедуру **sp_columns**:

```
[exec] sp_columns @table_name [, @table_owner]
                [, @table_qualifier] [, @column_name]
```

sp_columns возвращает информацию о столбцах одной и более таблиц. Допускает использование групповых символов.

Параметры:

- **@table_name** – имя таблицы. Допускает групповые символы (для оператора **LIKE**)
- **@table_owner** – владелец таблицы. Допускает групповые символы
- **@table_qualifier** – имя базы данных. Допустимые значения - имя текущей базы данных или NULL
- **@column_name** – имя столбца. Допускает групповые символы

11. Переименование таблиц.

Для переименования таблиц, а также других объектов БД, служит системная сохраненная процедура **sp_rename**:

```
[exec] sp_rename @objname, @newname[, @indorcol]
```

Вы должны быть владельцем объекта, который хотите переименовать. DBO может переименовывать любой пользовательский объект. Кроме того, переименовываемый объект должен находиться в текущей базе данных.

Параметр **@indorcol** используется, если нужно переименовать индекс или столбец таблицы и четко указать, что переименовывается (например, если имя индекса совпадает с именем столбца). Этот параметр может принимать значения **'index'** и **'column'**. Если параметр **@indorcol** не указан или равен NULL, процедура попытается определить тип переименовываемого объекта автоматически. В любом случае, для переименования столбца или индекса параметр **@objname** должен содержать имя таблицы и имя переименовываемого объекта, разделенные точкой.

Чтобы переименовать саму базу данных, используйте процедуру **sp_renamedb**.

Для ASE версий младше 15.0 при переименовании идентификатор объекта не меняется, поэтому все связанные с переименованным объектом процедуры, триггеры, представления продолжают работать по-старому. Однако во избежание труднообъяснимых сбоев, при переименовании объекта все связанные с ним процедуры, триггеры или представления должны быть изменены или просто пересозданы.

Особенно это важно, когда один объект переименовывается (таблица xxx в ууу) и тут же создается новый объект с именем старого (новая таблица xxx). Процедуры, которые не были пересозданы, будут ссылаться на таблицу ууу, а не на xxx.

Для ASE 15.0 и старше при переименовании объектов связи изменяются автоматически.

Список зависимых объектов можно получить, вызвав процедуру **sp_depends**.

Пример использования sp_rename

```
-- переименование таблицы
exec sp_rename Clients, ClientsNew

-- переименование столбца
exec sp_rename "Clients.LastName", Family
-- или
exec sp_rename "Clients.LastName", Family, 'column'
```